

420-635-AB – NETWORK INSTALLATION AND
ADMINISTRATION I

PROJECT 3

APACHE SERVER OPTIMIZATION, SCRIPTING, AND SECURE CONFIGURATION

PREPARED BY: GUILLERMO PADILLA KEYMOLE

John Abbott College | Network Administration Program

Table of Contents

Introduction	7
TASK 1: Multi-Process Module (MPM)	8
Creating the Web Directory	8
Creating the Project Homepage	9
Setting the Default Web Directory and Homepage.....	9
Enabling mpm_prefork_module	10
Configuring MPM in httpd.conf	10
Testing Apache Configuration.....	10
Restart and Verify MPM Type	11
Validation: Testing Apache Homepage	12
TASK 2: APACHE SERVER MONITORING	13
Enabling Apache Monitoring Modules	13
Enabling ExtendedStatus	13
Configuring Access to Monitoring Pages	14
Adding Task 2 Hyperlinks to Homepage	14
Validation 2: Apache Monitoring Pages	15
Validation from an Authorized IP.....	15
Validation from Unauthorized IP	16
TASK 3: Common Gateway Interface (CGI)	17
Preparing the CGI Directory	17
Creating the Perl CGI Script	17
Enabling CGI Support in Apache	17
Updating the Homepage	18
Validation Task 3: CGI Script Execution	18
Validation Using Web Browser	18
Validation Using curl	19
TASK 4 – PHP	20
Installing and Preparing PHP	20

Creating the PHP Script Directory	20
Writing the PHP Script.....	21
Preparing the Homepage Links	21
Preparing the Network for Testing.....	22
Restarting Services	22
Validation 4: PHP Script Output	23
Positive Validation from 192.168.100.0/24 Subnet	23
Positive Validation from Another Subnet.....	23
TASK 5: MySQL and PHP Integration	24
Installing MySQL Server and PHP MySQL Support	24
Setting Up the Database and Table	24
Inserting Employee Records	26
Creating a MySQL User and Granting Privileges	26
Creating the PHP Script.....	26
Allow Apache to Connect to MySQL (SELinux).....	27
Restarting Services	28
Creating a Specific MySQL User.....	28
Adding Hyperlink to Homepage	28
TASK 5 Validation: Displaying the Employees Table from MySQL	29
Accessing the PHP Script through the Homepage	29
Viewing the Employees List	30
Task 6: SSL.....	31
The goal of this task is to secure the Apache web server with SSL (Secure Sockets Layer) . SSL encrypts the communication between the client (web browser) and the server to ensure confidentiality and data integrity.....	31
Install the SSL Module for Apache.....	31
Generate a Private Key and SSL Certificate	31
Configure Apache SSL Settings.....	32
Restart Apache.....	33
Validation: SSL (Task 6)	33

Accessing the SSL Secured Website	33
Browser Security Warning	33
Accepting the Risk	34
Confirming Successful SSL Connection	34
Conclusion	35

Table of Figures

Figure 1 Project Part III Overview	7
Figure 2 Creating and preparing /var/www/html_project3 with correct permissions and SELinux context	8
Figure 3 Setting permissions, SELinux, and opening firewall services for HTTP/HTTPS	8
Figure 4 Editing the master project index page (master_project3.html)	9
Figure 5 Updated Apache DocumentRoot and DirectoryIndex to point to the project folder and homepage	9
Figure 6 Enabling mpm_prefork_module in 00-mpm.conf	10
Figure 7 Apache prefork MPM configuration block in httpd.conf	10
Figure 8 Verifying Apache is using prefork MPM	11
Figure 9 Apache running processes under prefork MPM	11
Figure 10 Project homepage successfully loading in browser from http://192.168.50.10/ ..	12
Figure 11 Enabling mod_status and mod_info in Apache module configuration	13
Figure 12 Adding ExtendedStatus to httpd.conf for detailed monitoring	13
Figure 13 Defining and restricting access to /server-status and /server-info	14
Figure 14 Adding root-relative hyperlinks for monitoring pages in master_project3.html ...	14
Figure 15 Accessing /server-status from 192.168.50.20	15
Figure 16 Accessing /server-info from 192.168.50.20	16
Figure 17 403 Forbidden responses from 10.35.16.1 subnet confirming restricted access	16
Figure 18 Preparing the /var/www/cgi-bin directory with correct permissions and SELinux context	17
Figure 19 Creating and setting executable permissions for task3.cgi	17
Figure 20 Apache configuration for CGI execution in httpd.conf	18
Figure 21 Adding Task 3 hyperlink to master_project3.html	18
Figure 22 Output of CGI script in browser confirming successful execution	18
Figure 23 Output of CGI script using curl	19
Figure 24 Installing PHP and php-fpm packages.	20
Figure 25 Enabling and starting the php-fpm service.	20
Figure 26 Creating the directory structure and preparing it with permissions and SELinux context	20
Figure 27 PHP script created to display different messages based on client IP.	21
Figure 28 Adding hyperlinks in the master project homepage for Task 4.	21
Figure 29 Adding an additional subnet address to the server network interface using nmcli.	22

Figure 30 Restarting php-fpm and Apache to apply the new settings	22
Figure 31 Validating PHP script from 192.168.100.0/24 subnet showing "Hello my friend!" message.....	23
Figure 32 Validating PHP script from another subnet showing "Hello stranger!" message..	23
Figure 33 Installing MySQL Server and PHP-MySQL driver.....	24
Figure 34 Starting and enabling the MySQL server.....	24
Figure 35 Creating the company database and employees table.	25
Figure 36 Inserting and verifying employee records inside the table.....	26
Figure 37 Creating a MySQL user root@server07 and granting permissions.	26
Figure 38 Preparing the directory with correct permissions and SELinux context	26
Figure 39 PHP script to display employees table from MySQL	27
Figure 40 Allowing Apache to connect to MySQL through SELinux.....	27
Figure 41 Restarting the necessary services to apply PHP-MySQL settings	28
Figure 42 Creating a dedicated user for better database access security	28
Figure 43 Adding hyperlink for Task 5 in the master homepage.	28
Figure 44 Navigating to the Task 5 hyperlink from the homepage	29
Figure 45 Viewing the Employees List fetched dynamically from the MySQL database	30
Figure 46 Installing the mod_ssl package using dnf.....	31
Figure 47 Creating a self-signed SSL certificate with OpenSSL and specifying certificate details.....	32
Figure 48 Setting ServerName to server07 in the SSL virtual host configuration	32
Figure 49 Specifying the SSL certificate and private key paths in ssl.conf	32
Figure 50 Restarting the Apache service to apply SSL changes	33
Figure 51 Accessing the secured version of the website via hyperlink	33
Figure 52 Browser warning about the self-signed SSL certificate	33
Figure 53 Option to accept the risk and continue to the website	34
Figure 54 Website successfully accessed via HTTPS (SSL secured)	34

Introduction

This project focuses on optimizing the performance and enhancing the functionality of an Apache web server on AlmaLinux. Building on the previous projects, this final part introduces key concepts in web server tuning, monitoring, secure communication, and dynamic content delivery using scripting languages.

The primary objectives of this project are to configure Apache's Multi-Processing Modules (MPMs) for better scalability, enable real-time monitoring features, and implement server-side scripts using both CGI and PHP. Additionally, the project integrates a MySQL database with PHP for dynamic content generation and configures SSL to secure client-server communications.

Each task is carefully tested using an Ubuntu client to simulate real-world scenarios. The project root is located at `/var/www/html_project3`, and all task-related files and scripts are accessible through a centralized homepage for easy validation.

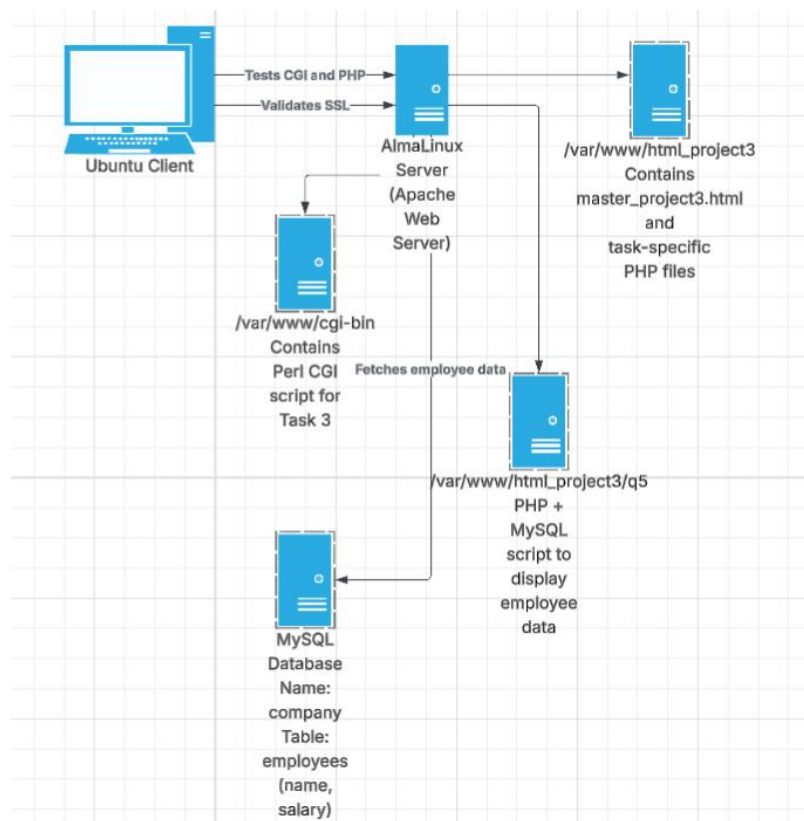


Figure 1 Project Part III Overview

TASK 1: Multi-Process Module (MPM)

The goal of this task was to improve the performance of the Apache web server by using the **prefork MPM**, which handles each request with its own separate process. This method avoids threads completely, making it a safer option for running things like CGI scripts or older applications.

To complete this task, I created the main project folder for the website, adjusted Apache's settings to control how many requests it can handle at once, enabled persistent connections to improve efficiency, and set time limits for how long the server should wait for client requests. These changes help the server work better when handling many users at the same time.

Creating the Web Directory

I created the base directory `/var/www/html_project3`, which will be used for all web-related content for this project. I also applied the correct permissions and SELinux context.

```
root@server07 ~ $ mkdir -p /var/www/html_project3
root@server07 ~ $ chmod -R 755 /var/www/html_project3/
root@server07 ~ $ chcon -R -t httpd_sys_rw_content_t /var/www/html_project3/
```

Figure 2 Creating and preparing `/var/www/html_project3` with correct permissions and SELinux context

I then enabled the SELinux boolean `httpd_unified` to allow Apache to read/write files in unified contexts:

```
root@server07 ~ $ setsebool -P httpd_unified 1
```

To allow external access to the server, I opened the default HTTP and HTTPS ports using **firewall services**:

```
root@server07 ~ $ firewall-cmd --permanent --add-service=http --zone=nm-shared
success
root@server07 ~ $ firewall-cmd --permanent --add-service=https --zone=nm-shared
success
root@server07 ~ $ firewall-cmd --reload
success
root@server07 ~ $
```

Figure 3 Setting permissions, SELinux, and opening firewall services for HTTP/HTTPS

Creating the Project Homepage

I created a file called `master_project3.html` to act as the homepage for all tasks in this project. This HTML file contains a list of all tasks with descriptions and hyperlinks for validation.

```
root@server07 ~ $ vim /var/www/html_project3/master_project3.html
```

Inside the file, the section for Task 1 reads:

```
<!DOCTYPE html>
<html>
<head>
  <title>Project Part 3</title>
</head>
<body>
  <h1>Project Part 3</h1>
  <hr>

  <h3>Task 1 - Multi-Process Module (MPM)</h3>
  <p>Apache server configured to handle 180 clients, with 12 processes, 6-12 idle threads, KeepAlive settings, and a request timeout of 55s.</p>
```

Figure 4 Editing the master project index page (`master_project3.html`)

Setting the Default Web Directory and Homepage

After creating the homepage file, I updated Apache's configuration so that it would serve all content from my project directory `/var/www/html_project3` instead of the default `/var/www/html`.

To do this, I opened the Apache configuration file:

```
root@server07 ~ $ vim /etc/httpd/conf/httpd.conf
```

Then I changed the `DocumentRoot` and added a new `<Directory>` block to match the new path. I also updated the default index page to `master_project3.html`:

```
DocumentRoot "/var/www/html_project3"
```

```
<Directory "/var/www/html_project3">
  Options Indexes FollowSymLinks
  AllowOverride None
  Require all granted
</Directory>
```

```
<IfModule dir_module>
  DirectoryIndex master_project3.html
</IfModule>
```

Figure 5 Updated Apache `DocumentRoot` and `DirectoryIndex` to point to the project folder and homepage

Enabling mpm_prefork_module

By default, Apache on AlmaLinux uses the event MPM. I needed to switch to the prefork module since it uses **processes only** and is more stable for certain configurations like CGI.

To do this, I edited the Apache module config:

```
root@server07 /etc/httpd/conf.modules.d $ vim 00-mpm.conf
```

In this file, I commented out the mpm_event_module and made sure the prefork module was loaded:

```
#LoadModule mpm_event_module modules/mod_mpm_event.so
LoadModule mpm_prefork_module modules/mod_mpm_prefork.so
```

Figure 6 Enabling mpm_prefork_module in 00-mpm.conf

Configuring MPM in httpd.conf

After enabling the module, I edited the main Apache config to apply the required performance parameters:

```
root@server07 ~ $ vim /etc/httpd/conf/httpd.conf
```

Inside the file, I added the following block:

```
<IfModule mpm_prefork_module>
    StartServers      12
    MinSpareServers   6
    MaxSpareServers   12
    MaxRequestWorkers 180
    ServerLimit       180
    ListenBacklog      100
</IfModule>

KeepAlive On
MaxKeepAliveRequests 50
KeepAliveTimeout 20

RequestReadTimeout header=20-40,MinRate=500 body=20-55,MinRate=500
```

Figure 7 Apache prefork MPM configuration block in httpd.conf

Testing Apache Configuration

Before restarting Apache, I verified that the configuration syntax was correct using the built-in validation command:

```
root@server07 ~ $ apachectl configtest
Syntax OK
root@server07 ~ $
```

Both `apachectl configtest` and `httpd -t` are used to check if the Apache configuration is valid. They both return Syntax OK if there are no errors.

The main difference is:

- `apachectl configtest` is a higher-level command that internally runs `httpd -t`, but it's safer to use because it also respects Apache's environment settings and behaves more like how the service actually runs.
- `httpd -t` is the direct, low-level command for syntax checking and is useful if you're debugging something very specific.

For this project, I used `apachectl configtest` because it's more beginner-friendly and better integrated with the system.

Restart and Verify MPM Type

After saving the changes, I restarted Apache and confirmed the MPM:

```
root@server07 /etc/httpd/conf.modules.d $ systemctl restart httpd
root@server07 /etc/httpd/conf.modules.d $ apachectl -V | grep -i mpm
Server MPM: prefork
```

Figure 8 Verifying Apache is using prefork MPM

Confirming Running Processes

To ensure the prefork configuration is in effect, I checked running Apache processes.

This showed multiple standalone `httpd` processes (no threads), confirming the prefork setup.

```
root@server07 /etc/httpd/conf.modules.d $ ps -ef | grep httpd
root      6852      1  0 09:25 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6855    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6856    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6857    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6858    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6859    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6860    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6863    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6864    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6889    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6890    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6891    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    6894    6852  0 09:26 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
root      6938    4539  0 09:42 pts/1    00:00:00 grep --color=auto httpd
```

Figure 9 Apache running processes under prefork MPM

What I Learned

In this task, I learned how to configure Apache's prefork MPM to handle multiple client requests using isolated processes, which is ideal for CGI and non-threaded workloads. I gained experience editing Apache's main configuration file to tune performance by setting process limits, enabling KeepAlive connections, and defining timeouts for client requests.

I also learned how to open HTTP and HTTPS access through the firewall using service names, how to apply the httpd_unified SELinux boolean for write permissions, and how to validate my configuration using `apachectl configtest` before restarting the server. This final step ensures that configuration errors do not cause service disruptions, a best practice in system administration.

Validation: Testing Apache Homepage

To confirm that all configurations were applied correctly, I opened a web browser on my Ubuntu client and visited the Apache server's IP address.

The `master_project3.html` homepage loaded successfully, showing links to each task in the project. This confirmed that:

- The DocumentRoot is set correctly
- Apache is serving from `/var/www/html_project3`
- The default page is set to `master_project3.html`
- Permissions, SELinux, and firewall rules are all allowing access



Figure 10 Project homepage successfully loading in browser from `http://192.168.50.10/`

TASK 2: APACHE SERVER MONITORING

The goal of this task was to configure Apache to allow real-time monitoring of server activity using the /server-status and /server-info pages. These pages help system administrators see how the server is performing, what resources are being used, and how requests are handled in real time. I also made sure these pages are only accessible from my internal subnet (192.168.50.0/24) for security reasons.

Enabling Apache Monitoring Modules

Apache includes two modules that provide monitoring features: mod_status and mod_info. I opened the module configuration file and made sure both modules were loaded:

```
root@server07 ~ $ vim /etc/httpd/conf.modules.d/00-base.conf
```

Inside the file, I verified that these lines were **uncommented**:

```
LoadModule status_module modules/mod_status.so
```

```
LoadModule info_module modules/mod_info.so
```

Figure 11 Enabling mod_status and mod_info in Apache module configuration

Enabling ExtendedStatus

To show more detailed information (like individual requests and CPU usage), I enabled the ExtendedStatus directive. I edited the main Apache config file:

```
root@server07 ~ $ vim /etc/httpd/conf/httpd.conf
```

And added this line:

```
ExtendedStatus On
```

Figure 12 Adding ExtendedStatus to httpd.conf for detailed monitoring

Configuring Access to Monitoring Pages

Still in `httpd.conf`, I added two blocks at the bottom of the file. These blocks define how the `/server-status` and `/server-info` pages work and restrict access only to users inside my subnet (192.168.50.0/24). This prevents unauthorized users from accessing sensitive server details:

```
<Location "/server-status">
    SetHandler server-status
    Require ip 192.168.50.0/24
</Location>

<Location "/server-info">
    SetHandler server-info
    Require ip 192.168.50.0/24
</Location>
```

Figure 13 Defining and restricting access to `/server-status` and `/server-info`

Adding Task 2 Hyperlinks to Homepage

To make it easier to test and validate the monitoring pages, I added links to `/server-status` and `/server-info` in my project homepage file: `master_project3.html`.

Initially, I tried using full URLs like:

```
<a href="http://192.168.50.10/server-status">
```

but this caused the browser to incorrectly generate URLs like:

```
http://192.168.50.10/192.168.50.10/server-status
```

To fix this, I changed the links to use root-relative paths, which correctly point to the server root no matter how the homepage is accessed. This approach worked reliably and ensured that clicking the links would correctly load Apache's monitoring pages from the project homepage.

```
<h3>Task 2 – Apache Server Monitoring</h3>
<ul>
  <li><a href="/server-status">To view the server status</a></li>
  <li><a href="/server-info">To view the server information</a></li>
</ul>
```

Figure 14 Adding root-relative hyperlinks for monitoring pages in `master_project3.html`

What I Learned

In this task, I learned how to activate Apache’s built-in monitoring features and make them secure by restricting access to a trusted subnet. I used the mod_status and mod_info modules, enabled detailed server activity with ExtendedStatus, and protected the monitoring endpoints using IP-based access control. This is a useful way to keep track of how the web server is running in real time without exposing sensitive information to the public.

Validation 2: Apache Monitoring Pages

After configuring the monitoring handlers and restarting Apache, I validated that both /server-status and /server-info were working as expected from the Ubuntu client (192.168.50.20), which is part of the allowed subnet (192.168.50.0/24).

Validation from an Authorized IP

From the Ubuntu client, I opened a browser to “192.168.50.10”. Both monitoring pages loaded successfully. The /server-status page displayed live server data, and /server-info returned detailed Apache configuration.

Task 2 – Apache Server Monitoring

To view the server status

To view the server information

Apache Status

192.168.50.10/server-status

Not secure

Apache Server Status for 192.168.50.10 (via 192.168.50.10)

Server Version: Apache/2.4.62 (AlmaLinux)
Server MPM: prefork
Server Built: Jan 10 2025 00:00:00

Current Time: Thursday, 24-Apr-2025 16:30:54 EDT
Restart Time: Thursday, 24-Apr-2025 16:14:50 EDT
Parent Server Config. Generation: 1
Parent Server MPM Generation: 0
Server uptime: 16 minutes 3 seconds
Server load: 0.08 0.16 0.11
Total accesses: 6 - Total Traffic: 275 kB - Total Duration: 15
CPU Usage: u.18 s.42 cu0 cs0 - .0623% CPU load
.00623 requests/sec - 292 B/second - 45.8 kB/request - 2.5 ms/request
1 requests currently being processed, 0 workers gracefully restarting, 11 idle workers

W

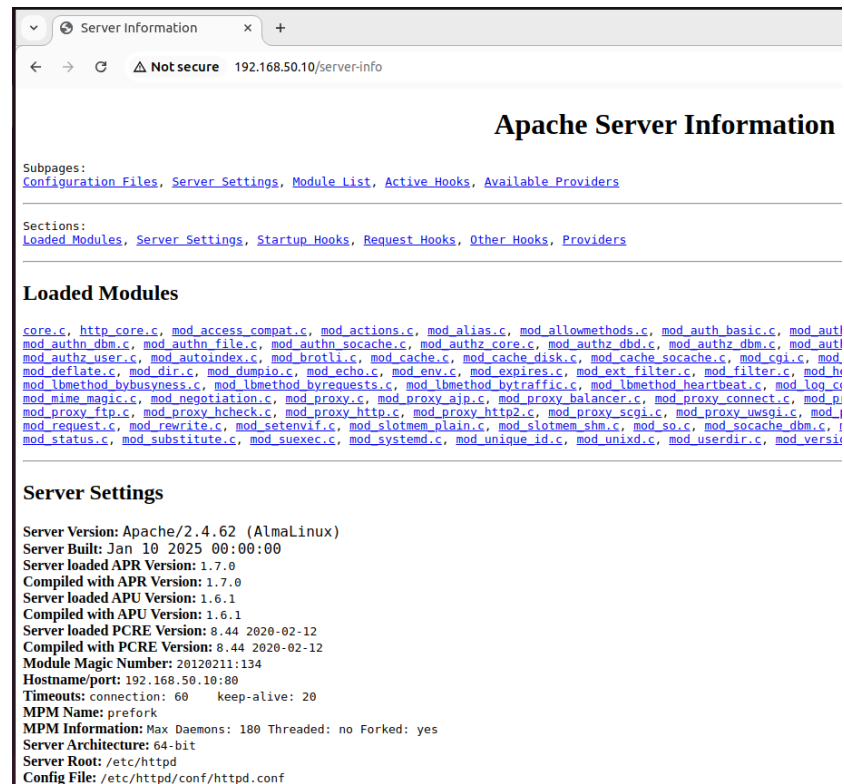
Scoreboard Key:
" " Waiting for Connection, "s" Starting up, "R" Reading Request,
"w" Sending Reply, "k" Keepalive (read), "D" DNS Lookup,
"c" Closing connection, "L" Logging, "G" Gracefully finishing,
"T" Idle cleanup of worker, "." Open slot with no current process

Srv	PID	Acc	M	CPU	SS	Req	Dur	Conn	Child	Slot	Client	Protocol	VHost	Request
0-0	9548	0/1/1	_	0.14	254	0	1	0.0	0.00	0.00	192.168.50.20	http/1.1	server07:80	GET / HTTP/1.1
2-0	9550	0/3/3	_	0.00	929	3	9	0.0	0.13	0.13	192.168.50.20	http/1.1	server07:80	GET /server-info HTTP/1.1
3-0	9551	0/1/1	_	0.15	196	0	0	0.0	0.00	0.00	192.168.50.20	http/1.1	server07:80	GET /server-status HTTP/1.1
5-0	9553	1/1/1	W	0.19	0	0	4	131.6	0.13	0.13	192.168.50.20	http/1.1	server07:80	GET /server-status HTTP/1.1

Figure 15 Accessing /server-status from 192.168.50.20

Task 2 – Apache Server Monitoring

- [To view the server status](#)
- [To view the server information](#)



The screenshot shows a web browser window with the title "Server Information" and the address bar displaying "192.168.50.10/server-info". The page content is titled "Apache Server Information" and includes links for "Subpages" (Configuration Files, Server Settings, Module List, Active Hooks, Available Providers) and "Sections" (Loaded Modules, Server Settings, Startup Hooks, Request Hooks, Other Hooks, Providers). The "Loaded Modules" section lists various modules like core.c, http_core.c, mod_access_compat.c, etc. The "Server Settings" section provides details such as "Server Version: Apache/2.4.62 (AlmaLinux)", "Server Built: Jan 10 2025 00:00:00", "Server loaded APR Version: 1.7.0", "Compiled with APR Version: 1.7.0", "Server loaded APU Version: 1.6.1", "Compiled with APU Version: 1.6.1", "Server loaded PCRE Version: 8.44 2020-02-12", "Compiled with PCRE Version: 8.44 2020-02-12", "Module Magic Number: 20120211:134", "Hostname/port: 192.168.50.10:80", "Timeouts: connection: 60 keep-alive: 20", "MPM Name: prefork", "MPM Information: Max Daemons: 180 Threaded: no Forked: yes", "Server Architecture: 64-bit", "Server Root: /etc/httpd", and "Config File: /etc/httpd/conf/httpd.conf".

Figure 16 Accessing /server-info from 192.168.50.20

Validation from Unauthorized IP

To ensure access was restricted as required, I temporarily modified my server's IP to 10.35.16.1/24, which is **outside the allowed subnet**.

```
root@server07 ~ $ nmcli connection modify LAN1 +ipv4.addresses 10.35.16.1/24
root@server07 ~ $ nmcli con down LAN1 ; nmcli con up LAN1
Connection 'LAN1' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/6)
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/7)
```

I then re-tested the monitoring pages, both returned, this confirms that Apache correctly denied access from unauthorized subnets.

```
root@server07 ~ $ curl -I http://10.35.16.1/server-status
HTTP/1.1 403 Forbidden
Date: Thu, 24 Apr 2025 20:46:49 GMT
Server: Apache/2.4.62 (AlmaLinux)
Content-Type: text/html; charset=iso-8859-1
root@server07 ~ $ curl -I http://10.35.16.1/server-info
HTTP/1.1 403 Forbidden
Date: Thu, 24 Apr 2025 20:46:53 GMT
Server: Apache/2.4.62 (AlmaLinux)
Content-Type: text/html; charset=iso-8859-1
```

Figure 17 403 Forbidden responses from 10.35.16.1 subnet confirming restricted access

TASK 3: Common Gateway Interface (CGI)

The objective of this task is to configure the Apache web server to support CGI scripts and execute a Perl script using the /cgi-bin directory. CGI allows the server to generate dynamic content by executing server-side scripts. This task demonstrates how to set up Apache to execute .cgi files using the Perl interpreter and display custom output in the browser.

Preparing the CGI Directory

Apache expects CGI scripts to reside in /var/www/cgi-bin. I first created the directory and applied the necessary permissions and SELinux context to allow Apache to execute scripts from it:

```
root@server07 ~ $ mkdir -p /var/www/cgi-bin
root@server07 ~ $ chmod -R 755 /var/www/cgi-bin
root@server07 ~ $ chcon -R -t httpd_sys_script_exec_t /var/www/cgi-bin
```

Figure 18 Preparing the /var/www/cgi-bin directory with correct permissions and SELinux context

Creating the Perl CGI Script

I then created a Perl script named task3.cgi with the following content. The script was saved in /var/www/cgi-bin/ and made executable.

```
root@server07 ~ $ vim /var/www/cgi-bin/task3.cgi

<h3>Task 3 - CGI</h3>
<p>Executes a Perl script displaying "This is Task 3 (CGI)".</p>
<ul>
  <li><a href="/cgi-bin/task3.cgi">To execute the CGI Script</a></li>
</ul>

root@server07 ~ $ chmod +x /var/www/cgi-bin/task3.cgi
```

Figure 19 Creating and setting executable permissions for task3.cgi

Enabling CGI Support in Apache

To enable CGI functionality, I ensured the mod_cgi module was loaded. I verified this by checking that the following line existed and was uncommented in the 00-base.conf file.

```
LoadModule cgi_module modules/mod_cgi.so
```

Next, I edited httpd.conf to include the appropriate directory configuration. These changes allow .cgi files to be executed as CGI scripts within the designated directory:

```
<Directory "/var/www/cgi-bin">
    Options +ExecCGI
    AddHandler cgi-script .cgi
    Require all granted
</Directory>
```

Figure 20 Apache configuration for CGI execution in httpd.conf

Updating the Homepage

To make the CGI task accessible from the main project index, I added the following link to master_project3.html:

```
<h3>Task 3 – CGI</h3>
<p>Executes a Perl script displaying “This is Task 3 (CGI)”.</p>
<ul>
  <li><a href="/cgi-bin/task3.cgi">To execute the CGI Script</a></li>
</ul>
```

Figure 21 Adding Task 3 hyperlink to master_project3.html

Validation Task 3: CGI Script Execution

After completing the CGI configuration, I tested whether Apache was correctly executing the Perl script placed in /var/www/cgi-bin.

Validation Using Web Browser

From the Ubuntu client (192.168.50.20), I opened a browser and visited “192.168.50.10”. After clicking the CGI hyperlink the page loaded successfully and displayed the following output:



Figure 22 Output of CGI script in browser confirming successful execution

Validation Using curl

I also validated the script using the command line. The output confirmed the script was executed correctly:

```
gkeymole@client07:~$ curl http://192.168.50.10/cgi-bin/task3.cgi  
<html><body><h2>This is Task 3 (CGI)</h2></body></html>gkeymole@client07:~$
```

Figure 23 Output of CGI script using curl

TASK 4 – PHP

The goal of this task is to configure Apache to support PHP using php-fpm and create a script that responds differently based on the client's IP address. Specifically, users from the 192.168.100.0/24 subnet will see a friendly message, while all others will receive a generic one.

Installing and Preparing PHP

On AlmaLinux, we are not using the traditional `mod_php` module. Instead, we are using the more modern and efficient php-fpm (FastCGI Process Manager), which allows PHP to run as a separate service and is better suited for high-performance environments.

Install PHP and php-fpm

This installed PHP 8.0 and related components, including php-fpm.

```
root@server07 ~ $ dnf install -y php
```

Figure 24 Installing PHP and php-fpm packages.

Enable and Start php-fpm Service

This command enables and immediately starts the php-fpm service.

```
root@server07 ~ $ systemctl enable --now php-fpm.service
Created symlink /etc/systemd/system/multi-user.target.wants/php-fpm.service → /usr/lib/systemd/system/php-fpm.service.
```

Figure 25 Enabling and starting the php-fpm service.

Note . After installing PHP, the system created `/etc/httpd/conf.d/php.conf`, which is responsible for integrating PHP-FPM with Apache. No manual module enabling is required in `00-base.conf` or `00-mpm.conf`.

Creating the PHP Script Directory

Create `/var/www/html_project3/q4` Directory. We created the project directory for Task 4, set correct permissions, and adjusted the SELinux context.

```
root@server07 ~ $ mkdir -p /var/www/html_project3/q4
root@server07 ~ $ chmod -R 755 /var/www/html_project3/q4
root@server07 ~ $ chcon -R -t httpd_sys_rw_content_t /var/www/html_project3/q4
```

Figure 26 Creating the directory structure and preparing it with permissions and SELinux context.

Writing the PHP Script

Write the PHP script

- Path: /var/www/html_project3/q4/task4.php
- Contents:

```
root@server07 ~ $ vim /var/www/html_project3/q4/task4.php
```

This script reads the client's IP address and responds based on their subnet.

```
<?php
$client_ip = $_SERVER['REMOTE_ADDR'];

if (preg_match('/^192\.168\.100\./', $client_ip)) {
    echo "<h2>Hello my friend!</h2>";
} else {
    echo "<h2>Hello stranger!</h2>";
}
?>
```

Figure 27 PHP script created to display different messages based on client IP.

Preparing the Homepage Links

Edit master_project3.html

We updated our homepage (/var/www/html_project3/master_project3.html) to include new links for Task 4:

```
<h3>Task 4 - PHP</h3>
<p>Displays a message based on user's IP subnet.</p>
<ul>
    <li><a href="http://192.168.100.1/q4/task4.php">To execute the PHP script using 192.168.100.1</a></li>
    <li><a href="http://10.35.16.1/q4/task4.php">To execute the PHP script using another subnet</a></li>
</ul>
```

Figure 28 Adding hyperlinks in the master project homepage for Task 4.

Preparing the Network for Testing

Add the 192.168.100.1 Subnet to the Server

To simulate two networks on the same VM, we added an extra IP address using nmcli. This method allows the server to listen on two different subnets.

```
root@server07 ~ $ nmcli connection modify LAN1 +ipv4.addresses 192.168.100.1/24
root@server07 ~ $ nmcli connection down LAN1 ; nmcli connection up LAN1
Connection 'LAN1' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/10)
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/11)
root@server07 ~ $
```

Figure 29 Adding an additional subnet address to the server network interface using nmcli.

Restarting Services

Restart php-fpm and httpd

After making configuration changes, we restarted both services in the correct order. Restarting php-fpm first ensures PHP handlers are ready before Apache starts using them.

```
root@server07 ~ $ systemctl restart php-fpm.service
root@server07 ~ $ systemctl restart httpd
root@server07 ~ $
```

Figure 30 Restarting php-fpm and Apache to apply the new settings

What I Learned

In this task, I learned how to configure Apache with PHP support using php-fpm rather than the older mod_php approach. I practiced writing a dynamic PHP script that checks the client's subnet to display customized messages.

Additionally, I learned how to modify network settings with nmcli to simulate multiple subnets on a server, how to adjust SELinux contexts properly, and how to update homepage hyperlinks to direct users to PHP scripts based on the server's IP addresses.

Validation 4: PHP Script Output

After setting up the PHP script and configuring the server, we validated the correct behavior depending on the client's subnet.

Positive Validation from 192.168.100.0/24 Subnet

Accessing the PHP Script via 192.168.100.1

While the AlmaLinux server was configured with an additional IP address 192.168.100.1, we opened a browser and clicked on the hyperlink. The page displays “Hello my friend!” because the client is detected as coming from the 192.168.100.0/24 subnet.

Task 4 - PHP

Displays a message based on user's IP subnet.

- [To execute the PHP script using 192.168.100.1](#)
- [To execute the PHP script using another subnet](#)

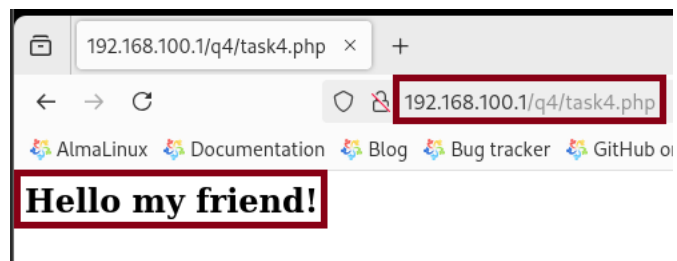


Figure 31 Validating PHP script from 192.168.100.0/24 subnet showing "Hello my friend!" message.

Positive Validation from Another Subnet

Accessing the PHP Script via 10.35.16.1

We also tested accessing the same PHP script from the server's other IP address. The page displays “Hello Stranger!” because the client IP does not match the 192.168.100.0/24 subnet.

Task 4 - PHP

Displays a message based on user's IP subnet.

- [To execute the PHP script using 192.168.100.1](#)
- [To execute the PHP script using another subnet](#)

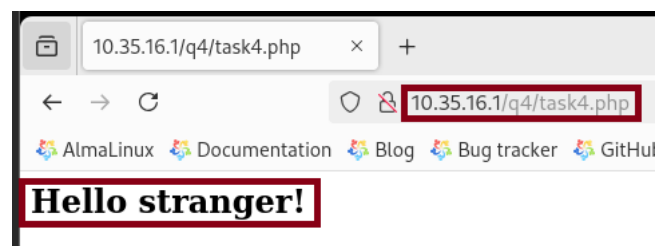


Figure 32 Validating PHP script from another subnet showing "Hello stranger!" message.

TASK 5: MySQL and PHP Integration

In this task, I connected a MySQL database with my Apache web server using PHP. The objective was to create a database called company, insert employees into it, then create a PHP script that displays those employees in a web browser.

Installing MySQL Server and PHP MySQL Support

To start, I installed the necessary packages.

The project requires MySQL, so I installed the official MySQL server, and also installed the PHP-MySQL module so that PHP can talk to MySQL databases.

```
root@server07 ~ $ dnf install -y mysql-server
```

```
root@server07 ~ $ dnf install -y php-mysqldb
```

Figure 33 Installing MySQL Server and PHP-MySQL driver

After installing, I enabled and started the MySQL service so it automatically starts on boot:

```
root@server07 ~ $ systemctl enable --now mysqld
Created symlink /etc/systemd/system/multi-user.target.wants/mysqld.service → /usr/lib/systemd/system/mysqld.service.
```

Figure 34 Starting and enabling the MySQL server

Setting Up the Database and Table

After MySQL was running, I connected to the MySQL shell. Inside the shell, I created a database named **company**. Inside the **company** database, I created an **employees** table with two fields: name and salary.

```
root@server07 ~ $ mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.41 Source distribution

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.01 sec)

mysql> create database company;
Query OK, 1 row affected (0.00 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| company |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)
```

```
mysql> use company;
Database changed
mysql> show tables;
Empty set (0.00 sec)

mysql>
mysql> create table employees(
    -> id int not null auto_increment,
    -> name varchar(100) not null,
    -> salary varchar(100) not null,
    -> primary key (id)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> show tables;
+-----+
| Tables_in_company |
+-----+
| employees |
+-----+
1 row in set (0.00 sec)

mysql> describe employees;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id | int | NO | PRI | NULL | auto_increment |
| name | varchar(100) | NO | | NULL | |
| salary | varchar(100) | NO | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Figure 35 Creating the company database and employees table.

Inserting Employee Records

I inserted several sample employee records into the table

```
mysql> insert into employees (name, salary) values ("Alexandre", "50000");
Query OK, 1 row affected (0.01 sec)

mysql> insert into employees (name, salary) values ("Blouin", "60000");
Query OK, 1 row affected (0.00 sec)

mysql> insert into employees (name, salary) values ("Charchar", "55000");
Query OK, 1 row affected (0.00 sec)
```

Figure 36 Inserting and verifying employee records inside the table.

Creating a MySQL User and Granting Privileges

As an extra step, I created a dedicated MySQL user to demonstrate best practices:

- This user can connect to the MySQL server from the hostname server07.
- The password for the user is alma.
- The user has full privileges, including the ability to grant privileges.

```
mysql> CREATE USER 'root'@'server07' IDENTIFIED BY 'alma';
Query OK, 0 rows affected (0.00 sec)

mysql> grant all privileges on *.* to 'root'@'server07' with grant option;
Query OK, 0 rows affected (0.01 sec)

mysql> flush privileges;
Query OK, 0 rows affected (0.00 sec)

mysql> EXIT;
Bye
root@server07 ~ $
```

Figure 37 Creating a MySQL user root@server07 and granting permissions.

Creating the PHP Script

The next step was to create a PHP script that connects to the MySQL database and displays the employee data inside an HTML table.

First, I prepared the directory where the script would be stored:

```
root@server07 ~ $ mkdir -p /var/www/html_project3/q5
root@server07 ~ $ chmod -R 755 /var/www/html_project3/q5
root@server07 ~ $ chcon -R -t httpd_sys_rw_content_t /var/www/html_project3/q5
root@server07 ~ $
```

Figure 38 Preparing the directory with correct permissions and SELinux context

Then I created the PHP script. In this script, I first connected to the database using the server's IP address and the credentials I set earlier. Then I fetched all records from the employees table and created a simple HTML table to display the data.

```
root@server07 ~ $ vim /var/www/html_project3/q5/task5.php

<?php
// Enable error reporting
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_reporting(E_ALL);

// Connection variables
$servername = "server07";
$username = "root";
$password = "alma";
$dbname = "company";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Query
$sql = "SELECT * FROM employees";
$result = $conn->query($sql);

// Display data
if ($result->num_rows > 0) {
    echo "<h2>Employees List</h2><table border='1'><tr><th>Name</th><th>Salary</th></tr>";
    while($row = $result->fetch_assoc()) {
        echo "<tr><td>" . htmlspecialchars($row["name"]) . "</td><td>$" . htmlspecialchars($row["salary"])
    . "</td></tr>";
    }
    echo "</table>";
} else {
    echo "0 results";
}

// Close connection
$conn->close();
?>
```

Figure 39 PHP script to display employees table from MySQL

Allow Apache to Connect to MySQL (SELinux)

Because SELinux security policies are enabled by default on AlmaLinux, Apache is not allowed to connect to remote databases unless we explicitly allow it.

I enabled the SELinux boolean `httpd_can_network_connect_db`. Without this step, the PHP script would fail to connect even if everything else was configured correctly.

```
root@server07 ~ $ setsebool -P httpd_can_network_connect_db 1
```

Figure 40 Allowing Apache to connect to MySQL through SELinux

Restarting Services

After changing permissions and SELinux settings, I restarted PHP-FPM and Apache to apply all changes:

```
root@server07 ~ $ systemctl restart php-fpm.service
root@server07 ~ $ systemctl restart httpd
root@server07 ~ $
```

Figure 41 Restarting the necessary services to apply PHP-MySQL settings

Creating a Specific MySQL User

To go a step beyond the minimum project requirements, I created a specific MySQL user that connects from the server's hostname (server07) instead of using the default root.

Inside MySQL, I ran:

```
mysql> CREATE USER 'root'@'server07' IDENTIFIED BY 'alma';
Query OK, 0 rows affected (0.00 sec)

mysql> grant all privileges on *.* to 'root'@'server07' with grant option;
Query OK, 0 rows affected (0.01 sec)

mysql> flush privileges;
Query OK, 0 rows affected (0.00 sec)

mysql> EXIT;
Bye
root@server07 ~ $
```

Figure 42 Creating a dedicated user for better database access security

Adding Hyperlink to Homepage

Finally, I updated my master_project3.html to include a link to my new Task 5 script:

```
root@server07 ~ $ vim /var/www/html_project3/master_project3.html

<h3>Task 5 – MySQL/PHP</h3>
<p>PHP script connects to MySQL to display employee records in a table.</p>
<ul>
  <li><a href="/q5/task5.php">View the Employees Table from MySQL</a></li>
</ul>
```

Figure 43 Adding hyperlink for Task 5 in the master homepage.

What I Learned

In this task, I learned:

- How to install and configure MySQL Server alongside an Apache and PHP environment.
- How to create and manage a database, insert data, and retrieve it through PHP.
- How to work with SELinux policies to allow network database connections.
- How to manage MySQL users and permissions securely.
- How to dynamically generate HTML content with PHP by querying a database.

This task was really important because it showed me how web servers and databases interact behind the scenes, a fundamental skill for any full-stack developer or server administrator.

TASK 5 Validation: Displaying the Employees Table from MySQL

After completing the configuration for Task 5, I validated that my PHP script correctly connected to the company database and displayed the contents of the employees table inside a web browser.

Accessing the PHP Script through the Homepage

I opened a web browser on my AlmaLinux Server. On the main project homepage (master_project3.html), I located the **Task 5** section and clicked on the link:

Task 5 - MySQL/PHP

PHP script connects to MySQL to display employee records in a table.

- [View the Employees Table from MySQL](#)

Figure 44 Navigating to the Task 5 hyperlink from the homepage

Viewing the Employees List

Clicking the hyperlink successfully opened the PHP script (task5.php). The page correctly displayed the list of employees I had inserted into the company database.

The table included the names and salaries as expected:

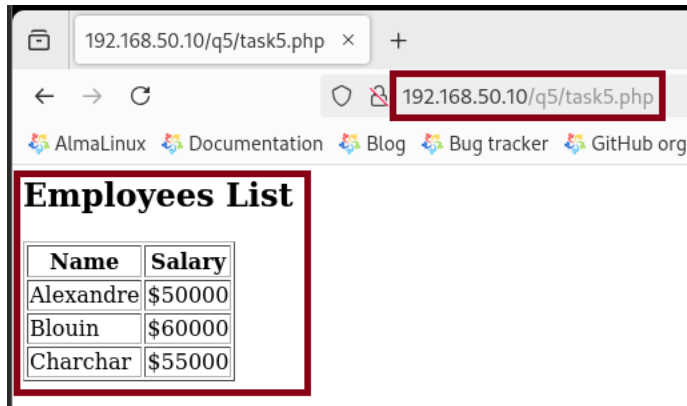


Figure 45 Viewing the Employees List fetched dynamically from the MySQL database

The validation shows that the integration between Apache, PHP, and MySQL is working perfectly.

The employees' data inserted into the database is correctly pulled and presented in an HTML table through the browser, as required by the project instructions.

Task 6: SSL

The goal of this task is to **secure the Apache web server with SSL (Secure Sockets Layer)**.

SSL encrypts the communication between the client (web browser) and the server to ensure confidentiality and data integrity.

Install the SSL Module for Apache

First, we need to install the SSL module (`mod_ssl`) to enable HTTPS support in Apache. This installs everything needed to allow Apache to handle secure HTTPS connections.

```
root@server07 ~ $ dnf install -y mod_ssl
```

Figure 46 Installing the `mod_ssl` package using `dnf`

Generate a Private Key and SSL Certificate

Next, we create a **self-signed SSL certificate** using OpenSSL.

This generates both the **private key** (kept secret) and the **public certificate** (shared with clients).

During the certificate creation, we entered the following details:

1. **Country Name (C)** → CA *(2-letter code for Canada)*
2. **State or Province Name (ST)** → Quebec (*Full province name*)
3. **Locality Name (L)** → Montreal (*City name*)
4. **Organization Name (O)** → John Abbott (*Educational institution*)
5. **Organizational Unit Name (OU)** → Left blank (*Optional field*)
6. **Common Name (CN)** → server07 (*Server hostname, matches future web address*)
7. **Email Address** → 6353322@johnabbottcollege.net (*Student email*)

This produced the two necessary files:

- `/etc/pki/tls/private/server.key` (Private key)
- `/etc/pki/tls/certs/server.crt` (Certificate)

[illegible]

Figure 47 Creating a self-signed SSL certificate with OpenSSL and specifying certificate details

Configure Apache SSL Settings

Now, we configure Apache to use the SSL certificate and key we generated.

We edit the default SSL configuration file.

```
root@server07 ~ $ vim /etc/httpd/conf.d/ssl.conf
```

Inside the `<VirtualHost _default_:443>` block, we make the following changes.

```
<VirtualHost _default_:443>

# General setup for the virtual host, inherited from global configuration
#DocumentRoot "/var/www/html"
ServerName server7:443
```

Figure 48 Setting ServerName to server07 in the SSL virtual host configuration

```

SSLCertificateFile /etc/pki/tls/certs/server.crt

# Server Private Key:
# If the key is not combined with the certificate, use this
# directive to point at the key file. Keep in mind that if
# you've both a RSA and a DSA private key you can configure
# both in parallel (to also allow the use of DSA ciphers, etc.)
# ECC keys, when in use, can also be configured in parallel
SSLCertificateKeyFile /etc/pki/tls/private/server.key

```

Figure 49 Specifying the SSL certificate and private key paths in `ssl.conf`

Restart Apache

After editing the SSL configuration, we restart Apache to apply the SSL settings:

```
root@server07 ~ $ systemctl restart httpd
```

Figure 50 Restarting the Apache service to apply SSL changes

Validation: SSL (Task 6)

After configuring SSL on the Apache server, we validated secure access (HTTPS) to the website by navigating to `https://server07` from the web browser.

Below are the steps and observations captured during validation:

Accessing the SSL Secured Website

- In the browser, we used the hyperlink created on the homepage labeled **"To view the website in SSL"**.
- Clicking on the link attempted to open the secured website using the HTTPS protocol.

Task 6 - SSL

View the secured version of the website using HTTPS.

- [To view the website in SSL](#)

Figure 51 Accessing the secured version of the website via hyperlink

Browser Security Warning

- Upon trying to access the site, a browser warning appeared, stating a potential security risk.
- This is expected behavior when using a **self-signed SSL certificate**, which is not trusted by browsers by default.



Warning: Potential Security Risk Ahead

Firefox detected a potential security threat and did not continue to **server07**. If you visit this site, attackers could try to steal information like your passwords, emails, or credit card details.

[Learn more...](#)

Go Back (Recommended)

Advanced...

Figure 52 Browser warning about the self-signed SSL certificate

Accepting the Risk

- To proceed, we clicked on **Advanced** and then **Accept the Risk and Continue**.
- This allowed the browser to load the website even with the self-signed certificate.

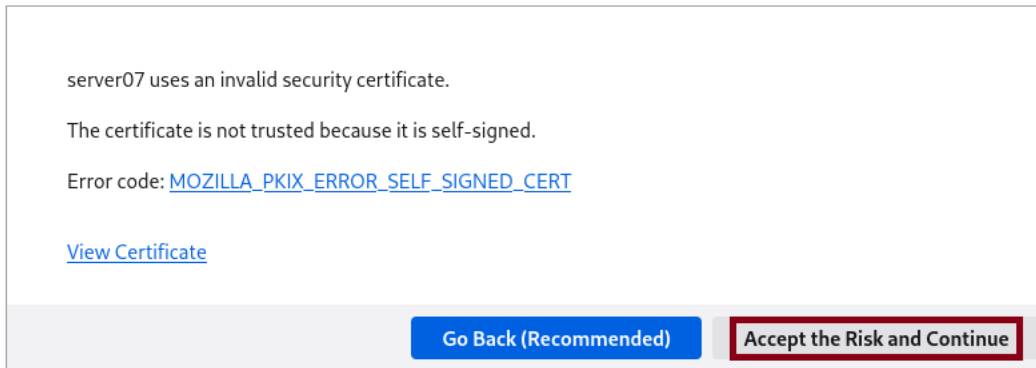


Figure 53 Option to accept the risk and continue to the website

Confirming Successful SSL Connection

- After accepting the warning, the secured website loaded successfully.
- The address bar showed the **HTTPS** protocol and a lock icon, confirming that SSL is active.

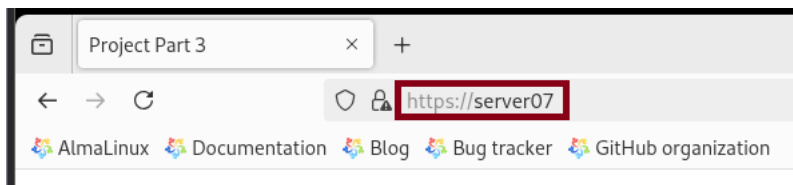


Figure 54 Website successfully accessed via HTTPS (SSL secured)

The validation confirms that:

- The SSL certificate was correctly installed and configured.
- The Apache web server can now securely serve content over HTTPS using the **server07** hostname.
- Browser warnings appeared because the certificate was self-signed, but access was successfully achieved after accepting the risk.

Conclusion

In this final part of the project, we successfully configured and optimized multiple services on the Apache server, demonstrating the ability to integrate advanced web technologies into a functional and secure environment.

Throughout the tasks, we completed the following:

- **Enabled server monitoring** using Apache's built-in status and info handlers, allowing us to monitor server performance and configuration details.
- **Implemented CGI scripts** by setting up the `/var/www/cgi-bin` directory and verifying Perl script execution through a web browser.
- **Configured PHP support** on the server, enabling the dynamic generation of web content based on client IP addresses using php-fpm and proper file permission settings.
- **Connected Apache to MySQL** by creating a database and developing a PHP script that displayed employee records in a formatted HTML table, ensuring seamless database interaction.
- **Secured the server with SSL** by generating a self-signed certificate and configuring HTTPS access, which allowed encrypted communication between clients and the server.

Each step of the configuration was carefully validated through browser tests, command-line queries, and structured screenshots to ensure all services operated correctly.

Troubleshooting steps were also followed whenever necessary, leading to a deeper understanding of how Apache modules, SELinux, PHP handlers, and SSL certificates interact.

Completing this project strengthened my skills in:

- Configuring Linux web services (Apache, PHP, MySQL).
- Implementing best practices for file permissions, SELinux policies, and server security.
- Testing and troubleshooting live server environments.
- Understanding real-world web hosting operations and client-server communication over secure protocols.

This project provided not only valuable technical practice but also a realistic view of the complexity and detail involved in setting up production-like web services.